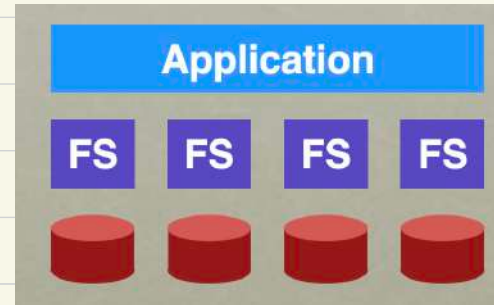


Redundant Arrays of Inexpensive Disks (RAIDs)

Sometimes, we want many disks — why?

we wish for more — capacity, reliability, and performance.

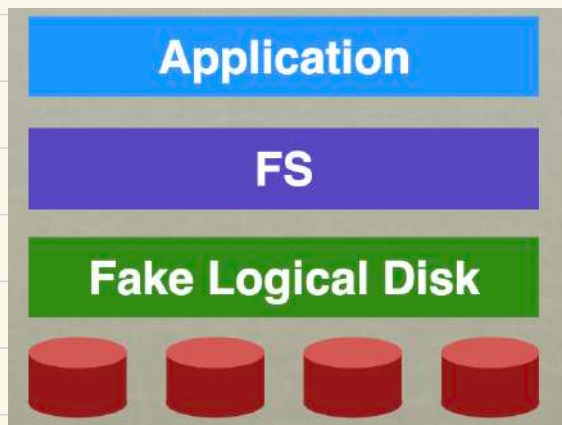
How to achieve this? **Solution 1:**
use a bunch of disks



Application is smart, stores different files on different systems.

Problem with the above approach? Upon a disk failure the data stored in the corresponding disk is lost.

Solution 2:



RAID: Redundant Array of Inexpensive Disks

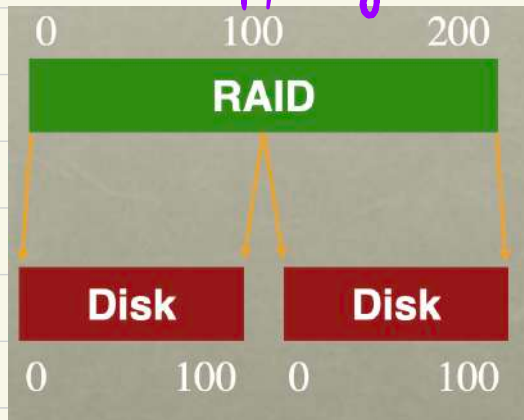
- use multiple disks in concert to build a faster, bigger, and more reliable disk system.
- externally, it looks like a disk: group of blocks one can read or write.
- data is spread across multiple disks with some form of "redundancy" → will see soon!

why inexpensive disks?

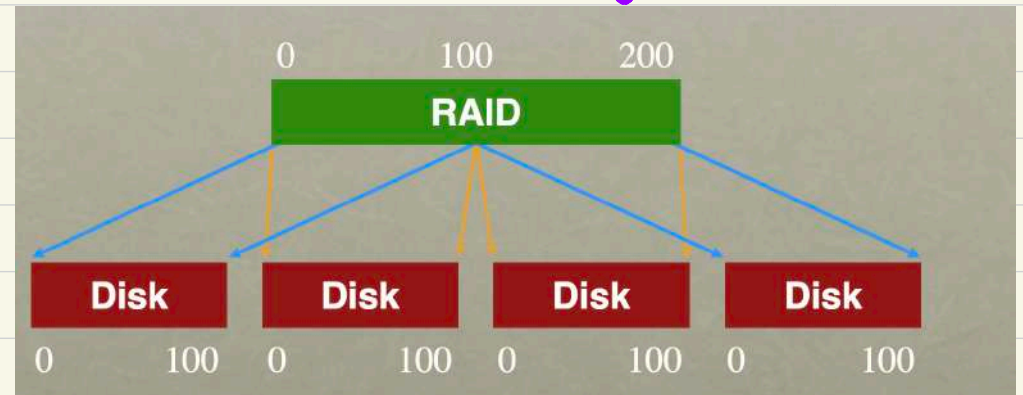
- * commodity disks cost less.
- * can buy many commodity H/W components for the same price as few high-end components.
- * strategy: write s/w to build high-quality logical devices from many cheap devices.
- * alternative to RAID → buy an expensive, high-end disk.

General strategy

mapping



redundancy



Build fast large disk from
smaller ones



add even more disks for
reliability

How does RAID work?

- when a file system issues a logical I/O request to the RAID,
the RAID internally
- * decides which disk (or disks) to access in order to complete the request
 - * issue one or more physical I/O to do so.

↳ nature of the above depends on the RAID level.
(will see soon!)

At a high level, a RAID is very much a specialized computer system
(it has a processor, memory, and disks inside).

Fault model: (what types of failures or faults we assume can happen in the system and how system behaves when they occur)
To understand RAID and compare different approaches, we have the following assumptions.

disks are fail-stop → each disk works or it doesn't
→ system knows when disk fails.

We are not worried about latent sector errors (where a single block is inaccessible) or silent data corruption.

How to evaluate a RAID?

Metrics involved

- **capacity**: given a set of N disks each with B blocks, how much useful capacity is available to the clients of RAID?
- **reliability**: how many disk faults can the given design tolerate?
- **performance** → two different metrics

Single-request latency
(to know how much parallelisms can exist during a single logical I/O operation)

Steady-state throughput
(total bandwidth of many concurrent requests)

two types of workloads.

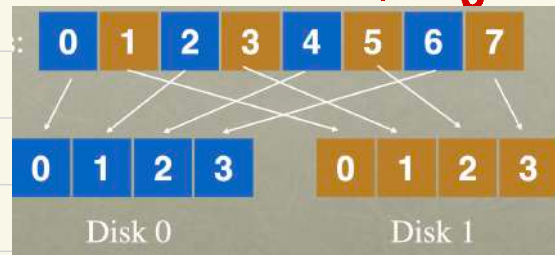
Sequential

(request to the array comes in large contiguous chunks. eg:- 1MB of data from block x to $(x+1MB)$)

random

(each request is small but from random locations on disk. eg:- 4KB from logical address 10, then address 550,000, then from 20,100, etc..)

RAID level 0: striping



→ stripe blocks across the disks of the system
i.e., it spreads the block of the array across the disks in a round-robin fashion.

designed to extract the most parallelism from the array when the requests are made for contiguous chunk of the array.

eg:- read blocks 4, 5, 6, 7.

	Disk 0	Disk 1	Disk 2	Disk 4
	0	1	2	3
stripe:	4	5	6	7
	8	9	10	11
	12	13	14	15

The blocks in the same row is called a **stripe**.

In the above example, only 1 block is placed in each disk in a stripe. However, it can be more as follows:

Chunk size = 2 blocks

	Disk 0	Disk 1	Disk 2	Disk 4
	0	2	4	6
	1	3	5	7
stripe:	8	10	12	14
	9	11	13	15

if 1 block is of size 4KB
⇒ the **chunk size** of this RAID array is 8KB (2 blocks)

⇒ a stripe consists of 4 chunks
or 32 KB data.

Small chunk size $\Rightarrow$ more parallelism of reads/writes to a single file
but more positioning time

big chunk size $\Rightarrow$ reduce positioning time (as a large portion fits within a disk)
but reduces intra-file parallelism.

so, deciding the "best chunk size" is challenging.

For the sake of convenience in the evaluation of RAIDs,

let $N = \# \text{ disks}$

$B = \text{capacity of 1 disk}$

$S(\text{MB/s}) = \text{sequential throughput of 1 disk}$

$R(\text{MB/s}) = \text{random throughput of 1 disk}$

$T(\text{ms}) = \text{latency of one small I/O operation.}$
time to complete

RAID-0 analysis

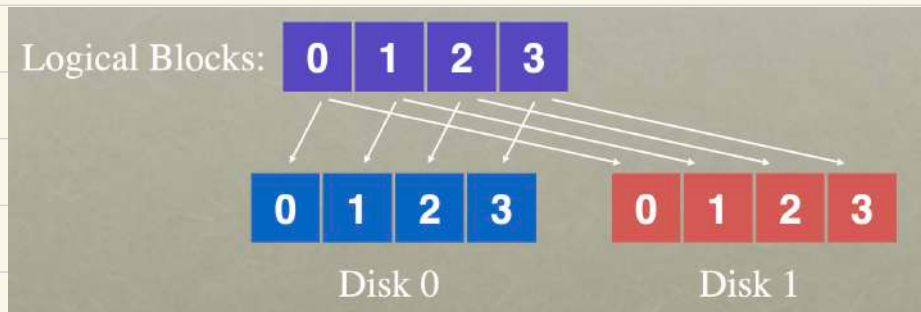
what is capacity? $N \times B$

How many disks can fail? 0 (as there is no redundancy of data)

latency: T

Throughput $\rightarrow$ sequential: $N \times S$ random: $N \times R$

RAID Level 1 : Mirroring



→ make more than one copy of each block in the system; each copy should be placed on a separate disk (to tolerate disk failures).

the way of mirroring in this figure is commonly known as RAID-10 or RAID 1+0, because it uses mirrored pairs + striping

	Disk 0	Disk 1	Disk 2	Disk 4
4 disks	0	0	1	1
	2	2	3	3
	4	4	5	5
	6	6	7	7

RAID-1 analysis

What is the capacity? $N/2 * B$

How many disks can fail? 1 (or may be even $N/2$)

latency → upon a single read request, RAID-1 direct the read to one of its copies

upon a single write request, it requires two physical writes, but both are done in parallel

⇒ latency is "almost" T itself.

→ think, why?

What about steady throughputs?

Sequential write $\rightarrow$ each logical write result in two physical writes

	Disk 0	Disk 1	Disk 2	Disk 4
4 disks	0	0	1	1
	2	2	3	3
	4	4	5	5
	6	6	7	7

eg:- to write in logical block 0, we need to write in both disk 0 and disk 1.

$\Rightarrow$ max. bandwidth for seq. writing = $\left(\frac{N}{2} * S\right)$
(half the peak bandwidth)

What about sequential read? $\rightarrow$ even though each logical read result in only one physical read (from any one of the disks) still the bandwidth could be $\left(\frac{N}{2} * S\right)$.

eg:- to read 0,1,2,3,4,5,6,7, we may issue the read

0 $\rightarrow$ D0, 1 $\rightarrow$ D1, 2 $\rightarrow$ D2, 3 $\rightarrow$ D3, 4 $\rightarrow$ D0, 5 $\rightarrow$ D1, 6 $\rightarrow$ D2, 7 $\rightarrow$ D3

$\Rightarrow$ each disk gets to serve only alternating requests (needs to skip in between, hence not utilizing full bandwidth).

What about random read? $\rightarrow (N \cdot R)$ (as sizes per location is small)

random write? $\rightarrow \left(\frac{N}{2} \cdot R\right)$ (as each logical write results in two physical writes)

RAID level 4: saving space with parity

RAID-4 combines striping for performance and parity for reliability.

	Disk 0	Disk 1	Disk 2	Disk 3 (Parity)
Stripe 0	A0	A1	A2	$P0 = A0 \oplus A1 \oplus A2$
Stripe 1	B0	B1	B2	$P1 = B0 \oplus B1 \oplus B2$
Stripe 2	C0	C1	C2	$P2 = C0 \oplus C1 \oplus C2$

Suppose you have N disks then

→ $(N-1)$ is used for data,
1 disk is reserved for parity.

Data blocks are striped across
data disks (like, RAID-0)

→ the parity disk stores XOR of corresponding data blocks.

What is the use of parity disk? If one disk fails then you can
reconstruct missing data using parity.

eg:- if Disk 1 fails then $A1 = A0 \oplus A2 \oplus P0$

This provides the tolerance for a single disk failure, like RAID-1,
but using less extra space.

RAID-4 analysis

capacity = $(N-1)B$

Sequential read? $(N-1) * S$

sequential write? $\sim (N-1) * S$ for large sequential writes (whole stripes)
parity can be computed efficiently and written
once per stripe.

Random read? $(N-1) * R$

what about random write?

To update a single block, we need to do the following.

- read old data block, C_{old}
- read old parity, P_{old}
- compute new parity (How? XOR all entries again $\rightarrow$ expensive in case of random)
better way $\rightarrow P_{new} = P_{old} \oplus C_{old} \oplus C_{new}$
- write new data
- write new parity

$\Rightarrow$ 4 I/O for one logical write

Also, parity disk becomes a bottleneck, since every write results in two I/O (one read, one write) on the parity disk.

$\Rightarrow$ throughput for random write = $R/2$, which is terrible!

latency $\rightarrow$ single read $\rightarrow T$

single write $\rightarrow$ 4 I/O (similar as above)

2 read
(can happen in parallel)

2 write
(can happen in parallel)

$\Rightarrow$ total latency
= twice that of a single disk.

RAID level 5: Rotating Parity

RAID-5 is almost the same as RAID-4 - except the parity blocks are spread (or distributed) across all disks instead of using a single dedicated parity disk.

↳ this removes the parity disk bottleneck that hurt RAID-4's random write performance.

	Disk 0	Disk 1	Disk 2	Disk 3
0	A0	A1	A2	P0
1	B0	B1	P1	B2
2	C0	P2	C1	C2
3	P3	D0	D1	D2

- each stripe has one parity block
- parity "rotates" among the disks to balance the load.
(as in RAID-4, parity = XOR of all data blocks in the stripe).

Sequential read = $(N-1) * S$

Sequential write $\approx (N-1) * S$

Random read $\approx N * R$ (all N disks are utilized for data reads, only a small fraction of each is used for parity)

Random write : (recall that in RAID-4, every write updated the same parity disk $\Rightarrow$ bottleneck!)

But here, parity updates are spread among disks, so the load is balanced.
 Still, each small write requires,

- read old data, C_{old}
- read old parity, P_{old}
- compute new parity
- write new data
- write new parity

$$P_{new} = C_{old} + P_{old} + C_{new}$$

$\Rightarrow$ 4 I/Os per write, but now distributed

$\therefore$ we can generally assume that given a large # random requests, we will be able to keep all disks about evenly busy.

$\Rightarrow$ total bandwidth = $\frac{N}{4} * R$

RAID-5 is the most common design of 4 RAID in market

RAID-0 $\rightarrow$ fastest, best capacity
 but at the cost of reliability

RAID-5 is better than RAID-1 for
 seq. workloads

RAID-1 " " " RAID-5 for
 random workloads.

	RAID-0	RAID-1	RAID-4	RAID-5
Capacity	$N \cdot B$	$(N \cdot B)/2$	$(N - 1) \cdot B$	$(N - 1) \cdot B$
Reliability	0	1 (for sure) $\frac{N}{2}$ (if lucky)	1	1
Throughput				
Sequential Read	$N \cdot S$	$(N/2) \cdot S^1$	$(N - 1) \cdot S$	$(N - 1) \cdot S$
Sequential Write	$N \cdot S$	$(N/2) \cdot S^1$	$(N - 1) \cdot S$	$(N - 1) \cdot S$
Random Read	$N \cdot R$	$N \cdot R$	$(N - 1) \cdot R$	$N \cdot R$
Random Write	$N \cdot R$	$(N/2) \cdot R$	$\frac{1}{2} \cdot R$	$\frac{N}{4} R$
Latency				
Read	T	T	T	T
Write	T	T	$2T$	$2T$